

Unit – III

(Operator Overloading)

Overloadable Operators: Definition, Overloadable operators, Syntax and rules of operator overloading, Unary operator overloading, Binary Operator overloading. Operator overloading with Member and non-member functions. Overloading binary operators using a friend function

Introduction to Operator Overloading

Definition:

Operator overloading allows C++ operators (e.g., +, -, ++, ==) to work with user-defined types (classes/structs) by defining custom implementations.

Purpose:

- Enable intuitive syntax for custom types

```
Complex c3 = c1 + c2; // Instead of c1.add(c2)
```

- Enhance code readability and expressiveness

Overloadable Operators

Operators	Examples
Arithmetic	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code>
Relational	<code>==</code> , <code>!=</code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code>
Assignment	<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code>
Unary	<code>++</code> , <code>--</code> , <code>!</code> , <code>-</code> (negate)
Subscript	<code>[]</code>
Function call	<code>()</code>
Stream I/O	<code><<</code> , <code>>></code>

Non-overloadable operators:

`::`, `*`, `.`, `:`, `?:`, `sizeof`, `typeid`

Syntax & Rules

Syntax:

```
ReturnType operator<symbol>(parameters) {  
    // Implementation  
}
```

Key Rules:

1. At least one operand must be user-defined type
2. Cannot change operator precedence/associativity
3. Cannot create new operators

Unary Operator Overloading

Program With Member Function:

```
class Counter {
    int count;
public:
    Counter(int c) {
        count = c;
    }

    // Prefix ++ (e.g., ++obj)
    Counter operator++() {
        ++count;
        return *this;
    }

    // Postfix ++ (e.g., obj++)
    Counter operator++(int) { // Dummy int parameter
        Counter temp = *this;
        count++;
        return temp;
    }

    void print() { cout << count; }
};

// Usage:
Counter c(5);
++c;    // c becomes 6
c++;    // Returns 6, then c becomes 7
```

Program With Friend Function

```
#include <iostream>
using namespace std;

class Counter {
    int count;
public:
    Counter(int c = 0) {
        count = c;
    } // Default parameter added

    // Friend declarations
    friend Counter& operator++(Counter& c); // Prefix ++
    friend Counter operator++(Counter& c, int); // Postfix ++

    void print() const {
        cout << count;
    }
};

// Prefix ++ (++obj)
Counter& operator++(Counter& c) {
    ++c.count;
    return c;
}

// Postfix ++ (obj++)
Counter operator++(Counter& c, int) {
    Counter temp = c;
    c.count++;
    return temp;
}

int main() {
    Counter c(5);

    cout << "Initial: ";
    c.print(); // 5

    // Test prefix
    Counter d = ++c;
```

```
    cout << "\nAfter ++c: ";  
    c.print(); // 6  
    cout << " | Returned value: ";  
    d.print(); // 6  
  
    // Test postfix  
    Counter e = c++;  
    cout << "\nAfter c++: ";  
    c.print(); // 7  
    cout << " | Returned value: ";  
    e.print(); // 6  
  
    return 0;  
}
```

Output:

```
Initial: 5  
After ++c: 6 | Returned value: 6  
After c++: 7 | Returned value: 6
```

Binary Operator Overloading:

```
#include <iostream>
using namespace std;

class Complex {
    double real;
    double imag;
public:
    // Constructor with initialization list
    Complex(double r = 0.0, double i = 0.0){
        real = r;
        imag = i;
    }

    // Operator + using this keyword
    Complex operator+(const Complex& other) {
        // Using this to access the current object's members
        double result_real = this->real + other.real;
        double result_imag = this->imag + other.imag;
        return Complex(result_real, result_imag);
    }

    void display() const {
        cout << real << " + " << imag << "i" << endl;
    }
};

int main() {
    Complex c1(2, 3);
    Complex c2(4, 5);

    Complex c3 = c1 + c2; // Uses operator+ with this

    cout << "c1: ";
    c1.display(); // 2 + 3i

    cout << "c2: ";
    c2.display(); // 4 + 5i

    cout << "c3 (c1 + c2): ";
```

```

    c3.display(); // 6 + 8i

    return 0;
}

```

Binary Operator With Friend Function

```

#include <iostream>
using namespace std;

class Complex {
    double real;
    double imag;
public:
    // Constructor with initialization list
    Complex(double r = 0.0, double i = 0.0) {
        real = r;
        imag = i;
    }

    // Friend function to overload the + operator
    friend Complex operator+(const Complex& c1, const Complex& c2);

    void display() const {
        cout << real << " + " << imag << "i" << endl;
    }
};

// Friend function definition
Complex operator+(const Complex& c1, const Complex& c2) {
    return Complex(c1.real + c2.real, c1.imag + c2.imag);
}

int main() {
    Complex c1(2, 3);
    Complex c2(4, 5);

    Complex c3 = c1 + c2; // Uses friend operator+

    cout << "c1: ";
    c1.display(); // 2 + 3i
}

```

```
cout << "c2: ";  
c2.display(); // 4 + 5i  
  
cout << "c3 (c1 + c2): ";  
c3.display(); // 6 + 8i  
  
return 0;  
}
```